



RESUME ANALYSER USING AI
A PROJECT REPORT

Submitted by

AKAASHZ M	717823L203
ASISHSURYA V K	717823L205
GUNA M	717823L217

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATIONS ENGINEERING

KARPAGAM COLLEGE OF ENGINEERING

COIMBATORE – 641 032

ANNA UNIVERSITY : CHENNAI 600 025

MAY 2026



**KARPAGAM COLLEGE OF ENGINEERING
COIMBATORE – 641 032**

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**RESUME ANALYSER USING AI**” is the Bonafide work of **AKAASHZ M (717823L203)**, **ASISHSURYA V K (717823L205)**, **GUNA M (717823L217)**” who carried out the project under my supervision.

SIGNATURE

Dr. R. Sarankumar, M.E., Ph.D,

HEAD OF THE DEPARTMENT

SIGNATURE

Ms. S. Apsara , AP/EC.,

SUPERVISOR

Electronics and Communication Engineering,
Karpagam College of Engineering,
Coimbatore- 641 032

Electronics and Communication Engineering,
Karpagam College of Engineering,
Coimbatore- 641 032

Certified that the candidate was examined during the viva voce examinations held on

Signature of the Internal Examiner with date

Signature of External Examiner with date

DECLARATION

We hereby declare that this Project report entitled “**RESUME ANALYSER USING AI**” submitted by me for the degree of **BE in Electronics and Communication Engineering at Karpagam College of Engineering, Coimbatore** is the record of original work done by me under the guidance and supervision of **Ms.S.APSARA AP/EC** at the Department of Electronics and Communications Engineering, Karpagam College of Engineering, Coimbatore – 641032 and has not formed the basis for the award of any degree, or diploma or titles in this institution or any other Institution of higher learning.

Date :

Place : Coimbatore

Name and Signature of the Candidate(s)

AKAASHZ M 717823L203

ASISHSURYA V K 717823L205

GUNA M 717823L217

ACKNOWLEDGEMENT

First and foremost praises and thanks to the almighty for her showers and blessings throughout my Mini project to complete it successfully.

I extend my gratitude to the Management of Karpagam College of Engineering, Coimbatore for the excellent infrastructure and support facilities to undergo the Mini project.

I am are very grateful to **Dr.P.KARTHIGA IKUMAR (Ph.D)**, the Principal and **Dr.R.SARANKUMAR,M.E.,(Ph.D), ELECTRONICS AND COMMUNICATION ENGINEERING** for provided the facilities, support and permission to carried out my Mini Project at our esteemed institution.

I record my sincere gratitude to my Project Coordinator **Mr.N.M.JAYADEVAN, M.E.,(Ph.D)** for giving inputs, encouragement for the continuous improvement during the progress and to complete this Mini project.

I would like to express my sincere gratitude to my Guide **Ms.S.APSARA AP/EC** for the continuous support for my UG study, for his motivation and adequate guidance which helped me to achieve success in all my accomplishments and to complete this Mini project.

I also thank all the teaching faculty members and non-teaching Staff members of the department of **ELECTRONICS AND COMMUNICATION ENGINEERING**, Karpagam College of Engineering, Coimbatore for their kindness and support.

I would like to thank **my parents, family members and friends** who sacrificed their time and energy to complete the Mini project successfully.

AKAASHZ M (717823L203)

ASISHSURYA V K (717823L205)

GUNA M (717823L217)

ABSTRACT

The project titled “RESUME ANALYZER USING AI” is an intelligent web-based application designed to evaluate and enhance the quality of resumes using modern technologies. In today’s competitive job market, crafting a well-structured and impactful resume is crucial for job seekers to stand out among numerous applicants. This project aims to simplify and automate the resume evaluation process by providing instant, accurate, and personalized feedback to users.

The system is developed using a combination of frontend and backend technologies, where the user uploads a resume in digital format. The application processes the document using Natural Language Processing (NLP) techniques to extract key information such as skills, education, experience, and keywords. It then compares the extracted data with industry standards and job-specific requirements to assess the effectiveness of the resume. The analyzer identifies missing keywords, improper formatting, grammatical errors, and lack of relevant skills, and provides suggestions for improvement.

Additionally, the system includes a scoring mechanism that rates the resume based on various parameters, enabling users to understand their strengths and areas that need enhancement. The user-friendly interface ensures accessibility and ease of use for individuals with varying levels of technical expertise. By leveraging automation, the Resume Analyzer reduces manual effort and provides quick insights, helping users optimize their resumes efficiently.

Overall, this project contributes to improving employability by guiding users in creating professional and job-oriented resumes.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGENO
	DECLARATION	i
	ACKNOWLEDGEMENT	ii
	ABSTRACT	iii
1.	INTRODUCTION	1
	1.1 BACKGROUND	2
	1.2 MOTIVATION	2
	1.3 PROBLEM STATEMENT	2
	1.4 OBJECTIVES	2
	1.5 SCOPE OF THE PROJECT	3
2.	COMPONENTS AND SPECIFICATION	4
	2.1 SYSTEM MODULES	4
	2.2 SYSTEM FEATURES	4
	2.3 SOFTWARE REQUIREMENTS	5
3.	METHODOLOGY	6
	3.1 METHODOLOGY STEPS	6
	3.2 COMPONENT SELECTION	8
	3.3 CIRCUIT DESIGN	9
	3.4 SOFTWARE DEVELOPMENT	9
	3.5 TESTING AND DEBUGGING	9
	3.6 PERFORMANCE EVALUATION	9
4.	BLOCK DIAGRAM	10
	4.1 SYSTEM FLOW	10
	4.2 FLOW OF OPERATION	11
5.	WORKING PRINCIPLE	13

6.	IMPLEMENTATION	15
	6.1 SOFTWARE IMPLEMENTATION	15
	6.1.1 RESUME UPLOAD MODULE	15
	6.1.2 RESUME PARSING MODULE	17
	6.1.3 NLP PROCESSING MODULE	21
	6.1.4 SCORING AND ANALYSIS MODULE	22
	6.1.5 FEEDBACK GENERATION MODULE	22
	6.2 SYSTEM WORKFLOW	23
	6.2.1 USER INPUT STAGE	23
	6.2.2 DATA EXTRACTION STAGE	30
	6.2.3 DATA PROCESSING STAGE	30
	6.2.4 EVALUATION STAGE	30
	6.2.6 OUTPUT GENERATION STAGE	30
7.	RESULTS	31
8.	FUTURE ENHANCEMENTS	33
9.	CONCLUSION	34
	REFERENCE	35
	APPENDIX	35

CHAPTER 1

INTRODUCTION

In today's competitive job market, resumes play a crucial role in determining a candidate's chances of getting shortlisted for interviews. Recruiters often rely on automated systems such as Applicant Tracking Systems (ATS) to filter large volumes of resumes efficiently. However, many candidates fail to meet these automated screening criteria due to lack of proper formatting, missing keywords, and ineffective presentation of skills.

To address this issue, this project proposes the development of an **AI-Based Resume Analyzer**, which evaluates resumes and provides intelligent suggestions for improvement. The system uses Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyze resume content, extract relevant information, and compare it with industry standards and job requirements. The application enables users to upload resumes, receive a detailed analysis report, and improve their resumes accordingly.

The project aims to simulate a real-time resume screening system used in recruitment industries, providing insights into resume quality, keyword optimization, and overall effectiveness. This system not only helps job seekers but also demonstrates the practical implementation of AI in recruitment automation.

1. Background

In recent years, recruitment processes have become increasingly automated due to the rise of digital platforms and online job applications. Organizations receive thousands of resumes for a single job opening, making manual screening inefficient and time-consuming. As a result, companies use ATS software to filter resumes based on keywords, formatting, and relevance.

Many candidates are unaware of these filtering mechanisms and often submit resumes that do not meet industry standards. This leads to rejection despite having the required qualifications.

2. Motivation

The motivation behind this project is to assist job seekers in creating professional, ATS-friendly resumes. Many students and fresh graduates lack guidance in resume preparation and fail to highlight their skills effectively. Additionally, manual resume review is not scalable and may lead to inconsistent evaluations.

By developing an AI-based Resume Analyzer, the process becomes automated, fast, and unbiased. The project is also motivated by the increasing use of AI in recruitment systems, making it essential for candidates to adapt to these technologies.

3. Problem Statement

In the current recruitment scenario, a large number of resumes are rejected due to poor formatting, lack of relevant keywords, and improper structure rather than lack of skills. Existing manual evaluation methods are inefficient and time-consuming, while many candidates do not have access to professional resume guidance.

Therefore, there is a need for an intelligent system that can automatically analyze resumes, identify weaknesses, and provide suggestions for improvement. The problem addressed in this project is to design and develop an AI-based system that evaluates resumes based on content quality, keyword relevance, formatting, and ATS compatibility.

4. Objectives

The main objectives of the AI-Based Resume Analyzer project are:

- To develop a web-based application for resume analysis.
- To extract key information such as skills, education, and experience using NLP techniques.
- To evaluate resumes based on ATS and industry standards.
- To provide a scoring system indicating resume quality.
- To suggest improvements in content, formatting, and keyword usage.
- To enhance user understanding of resume optimization techniques.
- To ensure a user-friendly and interactive interface.

5. Scope of the Project

The scope of this project includes analyzing resumes in digital formats such as PDF and Word documents. The system focuses on extracting and evaluating important components such as skills, education, experience, and keywords. It provides feedback and recommendations to improve resume quality and increase employability.

This application is intended for students, fresh graduates, and professionals seeking job opportunities. While the system provides automated analysis, it does not replace human recruiters but acts as a supportive tool. Future enhancements may include job-role-based analysis, resume-job matching, and integration with job portals.

CHAPTER 2

COMPONENTS AND SPECIFICATION

2.1 SYSTEM MODULES

- **Frontend (User Interface)**

Developed using React.js, it allows users to upload resumes and view analysis results.

- **Backend Server**

Built using Python ,it handles file processing and communication between frontend and AI modules.

- **AI/NLP Engine**

Processes resume text using NLP techniques to extract information and analyze content.

- **Database (Optional)**

Stores user data, resumes, and analysis reports.

- **File Processing Module**

Extracts text from PDF/DOCX files using libraries.

2.2 SYSTEM FEATURES

1. Resume Upload and Parsing

The system allows users to upload resumes in formats such as PDF and DOCX. Once uploaded, the file is processed using parsing techniques to extract readable text. This ensures that the system can analyze the content regardless of the document format.

2. Keyword Extraction and Matching

The analyzer identifies important keywords such as technical skills, tools, and domain-specific terms using NLP techniques. These keywords are then compared with industry standards or job descriptions to determine whether the resume is optimized for recruitment systems.

3. ATS Compatibility Check

The system evaluates whether the resume meets Applicant Tracking System (ATS) requirements. It checks for proper formatting, use of standard headings, keyword presence, and readability to ensure the resume can pass automated screening processes.

4. Resume Scoring System

A scoring mechanism is implemented to rate the resume based on multiple factors such as content quality, keyword relevance, formatting, and structure. This score helps users understand the overall effectiveness of their resume.

5. Grammar and Formatting Analysis

The system analyzes the resume for grammatical errors, spelling mistakes, and formatting inconsistencies. It ensures proper sentence structure, alignment, and readability, which are essential for professional presentation.

6. Skill Gap Identification

The analyzer compares the user's skills with required industry or job-specific skills. It identifies missing or weak areas and suggests additional skills that can improve the candidate's profile.

2.3 SOFTWARE REQUIREMENTS

- Frontend: React.js, HTML, CSS
- Backend: Python
- Libraries: NLP (scikit, NLTK), PDF parser, Fast API, Pandas, Numpy

CHAPTER 3

METHODOLOGY

The methodology of the AI-Based Resume Analyzer project focuses on designing a systematic and intelligent workflow to analyze resumes and provide meaningful feedback. The system follows a structured pipeline that begins with resume upload and ends with result generation. The entire process integrates Natural Language Processing (NLP), text extraction techniques, and rule-based as well as AI-driven analysis.

Initially, the user uploads a resume in PDF or DOCX format through the web interface. The uploaded document is then passed to a file processing module, where the content is extracted and converted into plain text. This extracted text is preprocessed by removing unwanted characters, stop words, and formatting inconsistencies to ensure accurate analysis.

Next, the cleaned text is processed using NLP techniques to identify and extract important entities such as skills, education, experience, certifications, and keywords. The extracted information is then compared with predefined datasets, industry standards, or job descriptions to evaluate the relevance and effectiveness of the resume.

Based on this comparison, the system calculates a score by considering multiple parameters such as keyword density, formatting quality, readability, and skill relevance.

3.1 Methodology Steps

The development of the Resume Analyzer follows a sequence of well-defined steps:

1. Problem Identification:

Understanding the challenges faced by job seekers in creating effective resumes.

2. Requirement Analysis:

Defining system requirements such as resume upload, text extraction, keyword analysis.

3. System Design:

Designing the architecture of the system, including frontend, backend, and AI modules.

4. Data Collection:

Collecting sample resumes and job descriptions to define keywords and evaluation.

5. Text Extraction:

Extracting textual data from uploaded resumes using parsing libraries.

6. Data Preprocessing:

Cleaning and structuring the extracted data by removing noise and irrelevant content.

7. NLP Processing:

Applying NLP techniques such as tokenization, part-of-speech named entity recognition.

3.2 Component Selection

Table 3.2 – Component Selection

Component	Function	Justification
Frontend Interface	React, JavaScript, Tailwind CSS	User interaction and resume upload.
API Communication	Axios, FastAPI	Connects frontend and backend
Text Extraction	Python	Converts PDF to text
Text Preprocessing	NLTK	Cleans and prepares text
Skill Extraction	Python, NLTK	Identifies skills
Similarity Calculation	Scikit-learn, NumPy	Matches resume with job role
Scoring & Categorization	Python, Pandas	Generates score and fit level

3.3 SYSTEM DESIGN

The system is designed using a three-tier architecture:

1. Presentation Layer (Frontend):

Allows users to upload resumes and view results.

2. Application Layer (Backend):

Handles business logic, data processing, and communication between modules.

3. Data Processing Layer (AI/NLP):

Performs text extraction, analysis, and scoring.

Data Flow:

User → Upload Resume → Text Extraction → NLP Processing → Analysis → Score → Feedback

3.4 Software Development

The software development process involves implementing all modules using appropriate technologies:

- **Frontend Development:** Built using React.js for interactive UI.
- **Backend Development:** Implemented using Python .
- **Text Extraction:** Libraries like PyPDF2 or docx are used.
- **NLP Processing:** Implemented using libraries such as NLTK and Fast API.

3.5 Testing and Debugging

- **Unit Testing:** Each module is tested individually.
- **Integration Testing:** Ensures all modules work together correctly.
- **Functional Testing:** Verifies system features such as upload and analysis.
- **Error Handling:** Checks system response for invalid inputs or failures.

3.6 Performance Evaluation

- **Accuracy:** Correct extraction of skills and keywords.
- **Processing Time:** Time taken to analyze a resume.
- **Scalability:** Ability to handle multiple users.

CHAPTER 4

BLOCK DIAGRAM

4.1 SYSTEM FLOW

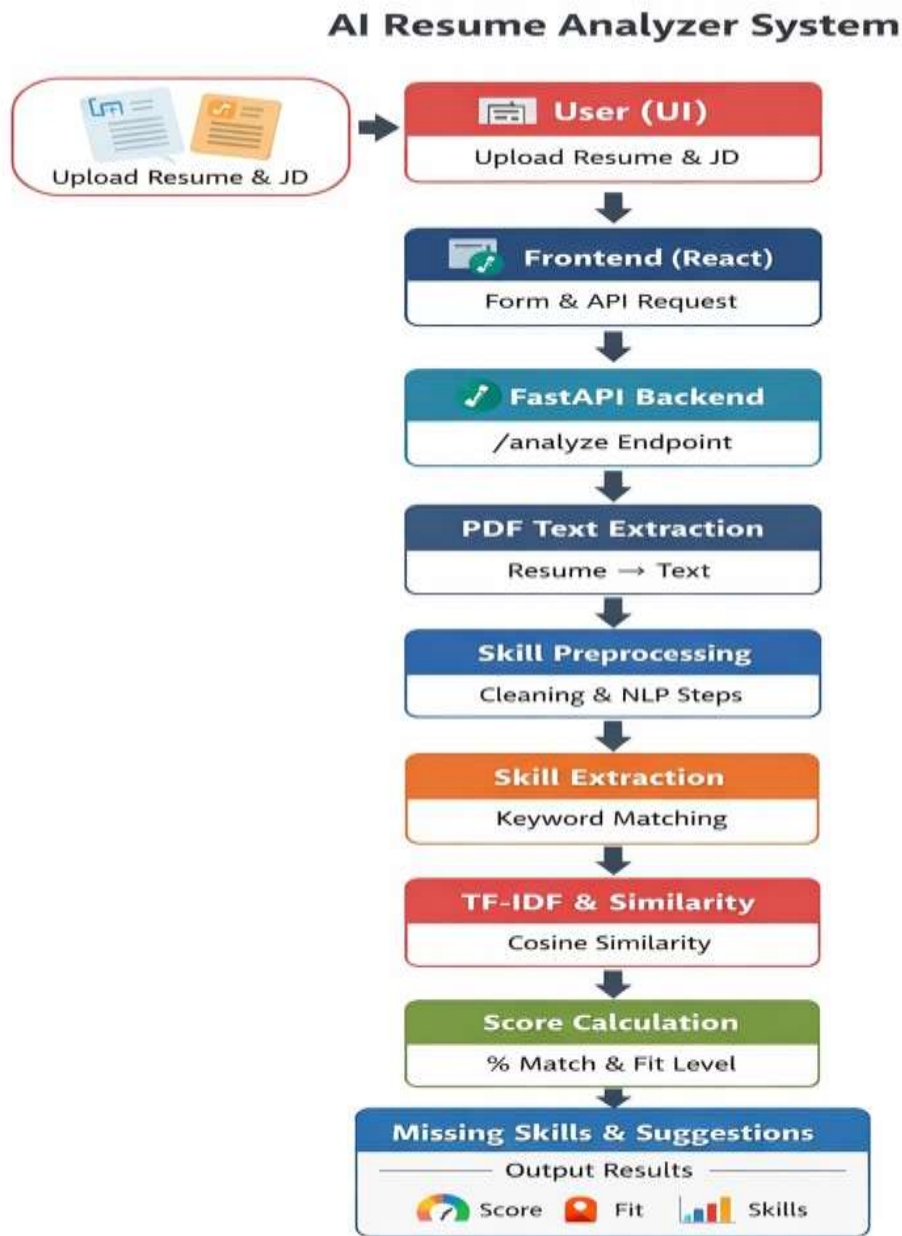


Fig 4.1 Block Diagram

1. User Interface (Input Stage):

The process starts when the user uploads a resume through the web application interface.

The interface supports formats such as PDF and DOCX and ensures easy interaction.

2. Resume Parsing Module:

Once the file is uploaded, the system extracts textual content from the document using parsing tools.

This step converts the resume into machine-readable text format.

3. Data Preprocessing:

The extracted text is cleaned by removing unwanted symbols, extra spaces.

This improves the accuracy of further analysis.

4. NLP Processing Module:

Natural Language Processing techniques are applied to analyze the text. This includes:

Tokenization (breaking text into words)

Part-of-Speech tagging

This step helps in identifying important information such as skills, education, and experience.

4.2 Flow of Operation:

The flow of operation of the AI-Based Resume Analyzer describes how the system processes a resume step by step to generate meaningful results.

Initially, the user uploads a resume through the web interface in formats such as PDF or DOCX. The system then extracts the textual content from the uploaded file using a parsing module. Once the text is extracted, it undergoes preprocessing where unnecessary characters, symbols, and stop words are removed to improve analysis accuracy.

After preprocessing, the cleaned text is analyzed using Natural Language Processing (NLP) techniques. During this stage, the system identifies key elements such as skills, education, experience, and important keywords present in the resume.

Next, the extracted information is compared with predefined industry standards or job-specific requirements. Based on this comparison, the system evaluates the quality of the resume and assigns a score considering factors like keyword relevance, formatting, and content quality.

The system analyzes sentence structure, grammar, and readability to ensure the resume is professionally written and easy to understand. The system identifies missing or weak skills by comparing the resume with job requirements and suggests improvements.

Finally, the system generates feedback, including suggestions for improvement, and displays the results on the user dashboard. This entire process happens quickly, providing users with real-time insights to enhance their resumes.

CHAPTER 5

WORKING PRINCIPLE

The working principle of the **AI-Based Resume Analyzer** is based on the integration of Artificial Intelligence (AI), Natural Language Processing (NLP), and text mining techniques to automatically evaluate and improve resumes. The system is designed to simulate the functioning of modern recruitment systems, particularly Applicant Tracking Systems (ATS), which are widely used by companies to filter and shortlist candidates.

When a user uploads a resume, the system initiates a sequence of operations that involve text extraction, preprocessing, analysis, and result generation. The primary goal of the system is to convert unstructured resume data into structured information and evaluate it against predefined standards.

Initially, the uploaded resume is processed using document parsing techniques to extract textual content. Since resumes can be in different formats such as PDF or DOCX, specialized libraries are used to ensure accurate text extraction without losing important information like headings, bullet points, and formatting structure.

After extraction, the text undergoes preprocessing, which includes removing special characters, converting text to lowercase, eliminating stop words, and correcting minor inconsistencies. This step is essential to prepare the data for efficient NLP processing.

The next stage involves Natural Language Processing, where the system applies techniques such as tokenization, part-of-speech tagging, and named entity recognition (NER).

These techniques help in identifying key components of the resume, such as:

- Personal information (optional fields)
- Educational qualifications
- Technical and soft skills
- Work experience
- Certifications and achievements

Once the relevant information is extracted, the system performs keyword analysis. It compares the extracted keywords with a predefined database of industry-relevant skills and job-specific requirements. This helps determine whether the resume is aligned with current market expectations.

Following keyword analysis, the system evaluates the resume using a scoring mechanism. The score is calculated based on multiple parameters, including:

- Keyword relevance and density
- Resume structure and formatting
- Content clarity and readability
- Presence of essential sections
- Grammar and language quality

Each parameter is assigned a weight, and the final score reflects the overall quality and effectiveness of the resume.

In addition to scoring, the system generates intelligent feedback. This includes suggestions for improving content, adding missing skills, correcting grammatical errors, and enhancing formatting. The feedback is personalized based on the specific issues identified in the resume.

The system also incorporates skill gap analysis, where it identifies missing competencies required for a particular job role. This feature helps users understand what additional skills they need to acquire to improve their employability.

Finally, the analyzed results, including the score and suggestions, are displayed on the user interface in a clear and structured format. The user can review the feedback and update their resume accordingly.

Overall, the working principle of the Resume Analyzer ensures an automated, efficient, and intelligent evaluation process. It reduces manual effort, increases accuracy, and provides real-time insights, making it a valuable tool for job seekers aiming to improve their resumes and succeed in the recruitment process.

CHAPTER 6

IMPLEMENTATION

6.1 HARDWARE IMPLEMENTATION

The hardware implementation of the LoRa-Based Wireless Telemetry System consists of two main sections: the Transmitter (Avionics/Flight Module) and the Receiver (Ground Station).

Both modules are integrated to ensure reliable, long-range communication and real-time monitoring of flight data.

6.1.1 Transmitter Section (Avionics Unit)

The transmitter module is the core of the flight system. It is built around a Raspberry Pi Pico microcontroller, which collects sensor data, processes it, and transmits it using the SX1278 LoRa module operating at 433 MHz.

Main components include:

- **Raspberry Pi Pico:** Acts as the central controller, reading sensor data and handling communication protocols.
- **BMP280 Sensor:** Measures atmospheric pressure and altitude.
- **MPU6050 Sensor:** Provides acceleration and angular velocity readings for flight motion analysis.
- **DHT11 Sensor:** Records temperature and humidity data.
- **MQ-2 Sensor:** Detects gas concentration in the environment.
- **SX1278 LoRa Module (Transmitter):** Sends encoded telemetry data to the ground station.
- **Power Supply:** A Li-ion battery powers all components for portability and flight endurance.

All sensors are connected to the Pico via I²C and analog pins, while the LoRa module communicates using SPI. The compact setup ensures low power consumption and stable data transmission during operation.

6.1.2 Receiver Section (Ground Station)

The receiver module is built using an Arduino UNO microcontroller, which receives telemetry packets from the transmitter through another SX1278 LoRa module and displays the decoded information on an LCD.

Main components include:

- **Arduino UNO:** Acts as the main processing unit for decoding and displaying telemetry data.
- **SX1278 LoRa Module (Receiver):** Receives the transmitted data packets from the flight module.
- **20x4 I²C LCD Display:** Displays altitude, temperature, pressure, and acceleration data in real time.
- **Power Supply:** Powered through a 5 V adapter or USB cable.

This setup forms the ground control station where operators can monitor flight parameters continuously.

6.1.3 Interfacing Summary

- **Transmitter Side:** Sensors → Raspberry Pi Pico → LoRa (Tx)
- **Receiver Side:** LoRa (Rx) → Arduino UNO → LCD Display

Both modules share a common ground and are powered by regulated 3.3 V/5 V supplies to ensure stable operation and prevent signal loss.

- The transmitter collects sensor data using Raspberry Pi Pico and sends it via LoRa Transmitter.
- The receiver obtains LoRa data, processes it using Arduino UNO, and displays results on LCD.

6.1.4 Project Setup

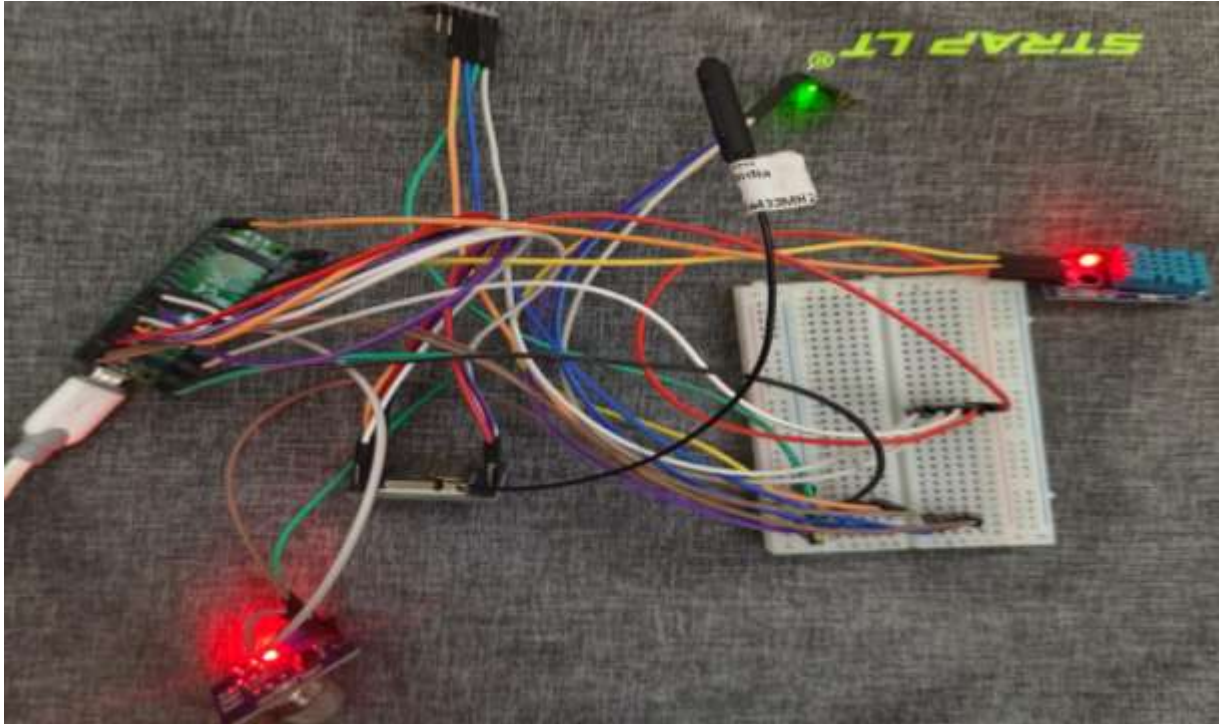


Fig 6.1 – Avionics System

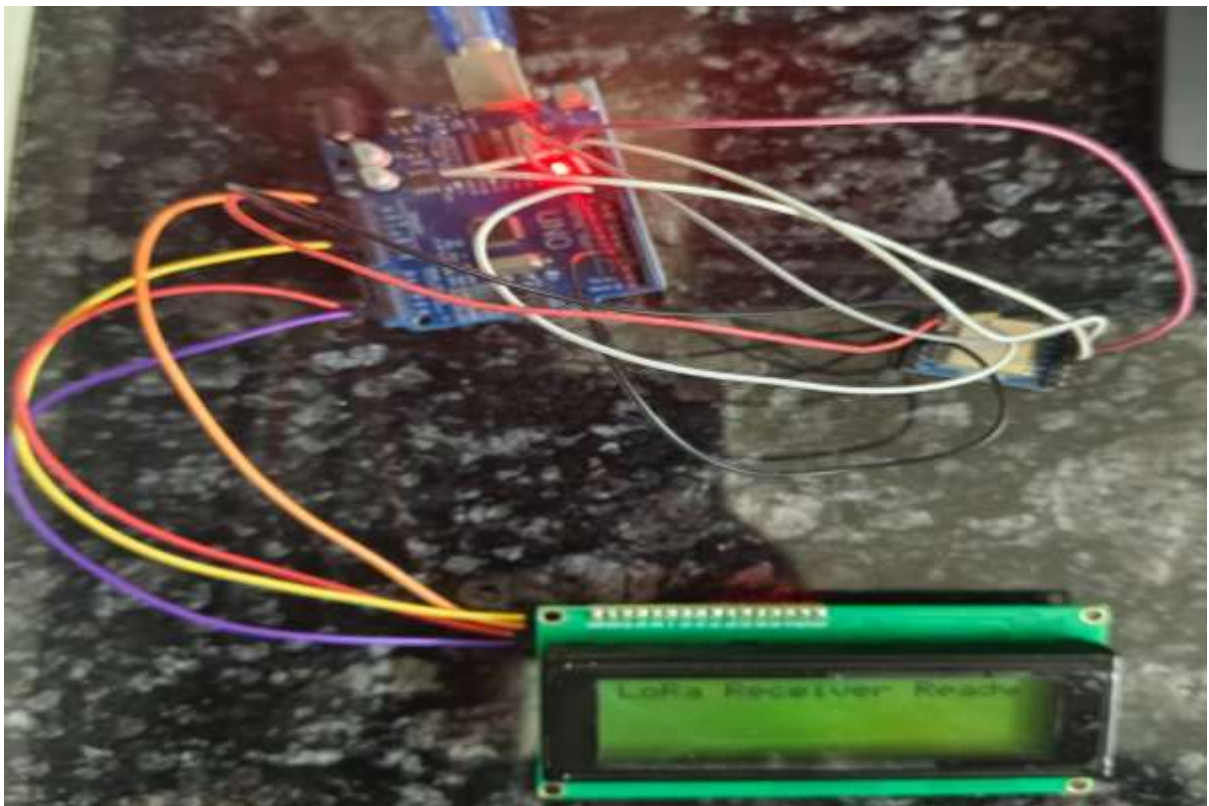


Fig 6.2 – Ground Station

6.1.4 SENSOR DATA OVER TIME

- **DHT11 – Temperature (°C)**

The temperature data shows minor fluctuations between 25°C to 29°C, simulating ambient temperature changes during flight. These variations may occur due to altitude changes or motor heat influence on the airframe.

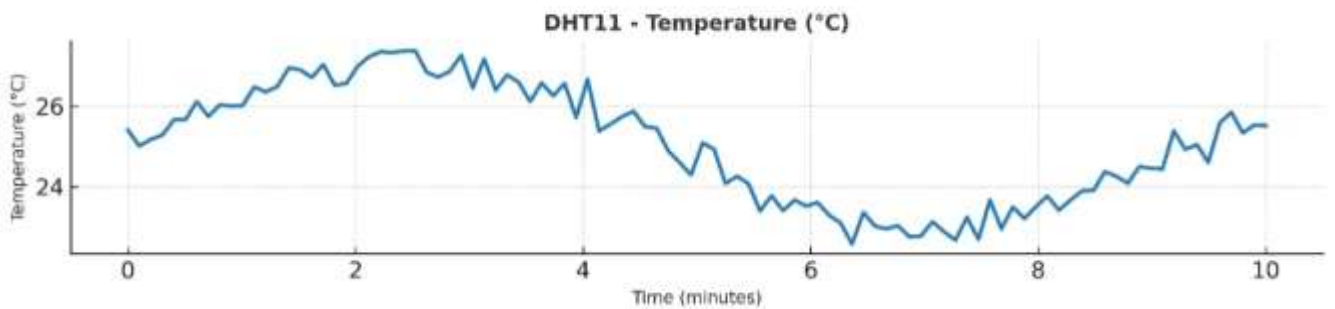


Fig 6.3 – DHT11 Sensor Graph

- **BMP280 – Altitude (m)**

The altitude readings gradually increase from around 100 m to 130 m, representing the RC aircraft's ascent and slight variations during flight.

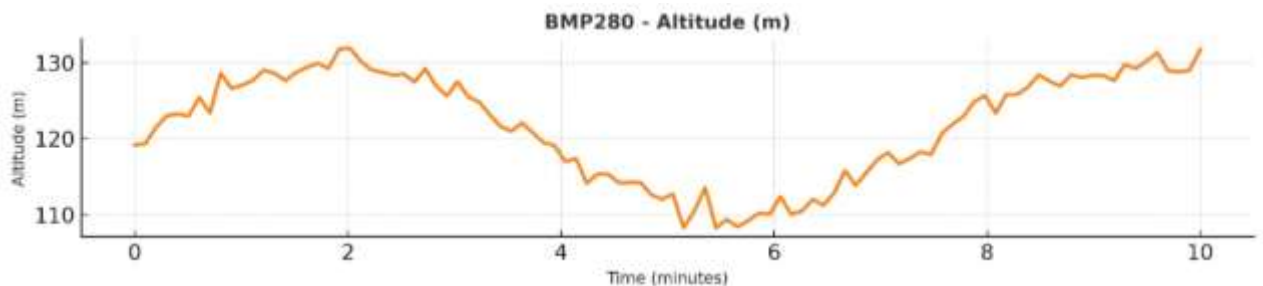


Fig 6.4 – BMP280 Sensor Graph

- **MPU6050 – Acceleration (m/s²)**

The acceleration curve shows small oscillations between 0.9–1.2 m/s², indicating changes in flight dynamics such as turning, climbing, or turbulence effects.

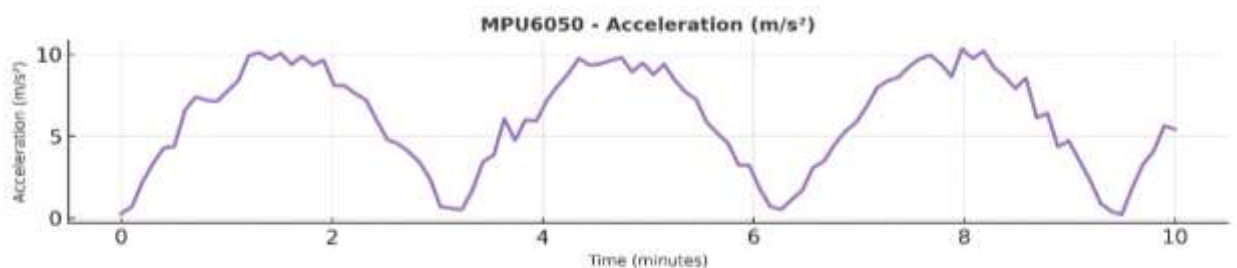


Fig 6.5 – MPU6050 Sensor Graph

▪ MQ-2 – Gas Concentration (ppm):

The gas concentration data varies slightly, simulating the detection of air quality or smoke presence during flight. It can also indicate exhaust gas detection or environmental monitoring capability.

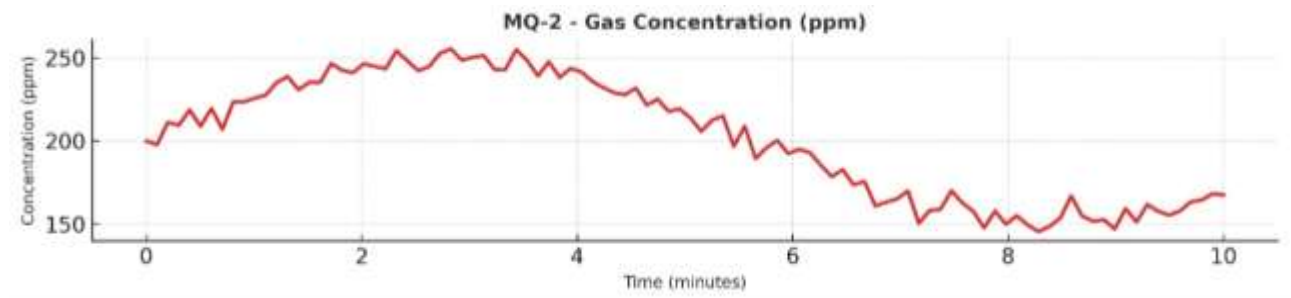


Fig 6.6 – MQ2 Sensor Graph

6.2 SOFTWARE IMPLEMENTATION

The software implementation of this project plays a crucial role in integrating all hardware components and ensuring seamless data flow between the transmitter (flight module) and receiver (ground station). The software is responsible for data acquisition, transmission, reception, decoding, and display operations in real time.

The development process involves programming both the Raspberry Pi Pico (Transmitter) and Arduino UNO (Receiver) using respective development environments — MicroPython/Arduino C for the Pico and Arduino IDE for the UNO.

6.2.1 Software Development Process

The software development process followed a structured approach, which includes:

- Requirement Analysis:

Identification of required functionalities such as sensor data acquisition, wireless data transmission, packet formation, and real-time display.

- System Design:

Logical data flow and communication between modules were planned. The system was divided into independent software modules — sensor interface, data encoder, transmitter, receiver, and display controller.

- **Integration:**

All modules were integrated into a single program for the transmitter and receiver. Communication between both units was tested and synchronized.

- **Testing and Debugging:**

Serial monitoring and LCD display output were used to verify sensor data accuracy and wireless communication reliability.

- **Validation:**

The final stage involved field testing to verify data transmission distance, signal stability, and packet integrity.

6.2.2 Transmitter Software (Raspberry Pi Pico)

The transmitter software is designed to continuously collect data from multiple sensors, process the information, and transmit it via the LoRa SX1278 module to the receiver. The program runs in a loop for real-time telemetry.

Main Functions:

- **Sensor Initialization:**

The program initializes the BMP280 (pressure), MPU6050 (acceleration and gyroscope), DHT11 (temperature/humidity), and MQ-2 (gas) sensors using I²C and analog interfaces.

- **Data Acquisition:**

Each sensor reading is periodically collected.

Example parameters:

- Temperature (°C)
- Humidity (%)
- Altitude (m)
- Pressure (hPa)
- Acceleration (m/s²)

➤ Gas concentration (ppm)

- Data Processing:

The values are processed, averaged, or filtered (if necessary) to reduce noise. Then, they are formatted into a structured data packet (e.g., CSV:

Temp,Hum,Alt,Press,AccX,AccY,AccZ).

- Data Transmission:

The packet is transmitted using the LoRa SX1278 transceiver configured via SPI.

Transmission occurs periodically (every 500 ms or 1 second).

- Error Handling:

If communication fails or a sensor returns invalid data, the program retries transmission or logs the error for debugging.

- Libraries Used:

- machine and time (MicroPython hardware control and timing)

- bmp280, mpu6050, dht, and custom LoRa libraries

- Sample Workflow:

Start → Initialize Sensors & LoRa → Read Data → Format Data → Transmit Packet
→ Wait → Repeat

6.2.3 Receiver Software (Arduino UNO)

The receiver software is responsible for receiving the transmitted packets, decoding the information, and displaying it on a 20×4 I²C LCD screen.

Main Functions:

- LoRa Initialization:

The Arduino initializes the SX1278 module using the SPI communication protocol. It sets parameters like frequency (433 MHz), spreading factor, and bandwidth.

- Listening for Packets:

The receiver remains in continuous listening mode, waiting for incoming LoRa data

packets from the flight module.

- **Packet Reception and Decoding:**

When a packet is received, it is read into a string buffer. The program parses the string and extracts each sensor value using delimiters (commas or spaces).

- **Data Display:**

The decoded data is displayed on the 20×4 I²C LCD in real time. Each line of the LCD represents a different parameter, such as:

- Line 1: Altitude and Pressure
- Line 2: Temperature and Humidity
- Line 3: Acceleration Data
- Line 4: Gas Concentration or Signal Strength

- **Serial Output (Optional):**

For debugging, the same data can also be printed to the serial monitor for logging.

- **Libraries Used:**

- SPI.h and LoRa.h — For wireless communication
- Wire.h and LiquidCrystal_I2C.h — For LCD interface
- String.h — For string manipulation and parsing

- **Sample Workflow:**

Start → Initialize LoRa & LCD → Wait for Packet → Receive Data → Decode Packet → Display → Repeat

6.2.4 Flow of Software Operation

Transmitter Side:

1. Power ON system.
2. Initialize sensors and LoRa transceiver.
3. Collect sensor data.
4. Format data packet.

5. Transmit data via LoRa.
6. Wait for a short delay.
7. Repeat process continuously.

Receiver Side:

1. Power ON ground station.
2. Initialize LoRa and LCD.
3. Listen for incoming packets.
4. Receive and decode data.
5. Display values on LCD.
6. Loop continuously for real-time updates.

6.2.5 Software Testing and Validation

- **Unit Testing:** Each sensor and module was individually tested to confirm proper functionality and data accuracy.
- **Integration Testing:** The transmitter and receiver were tested together to validate data transmission and synchronization.
- **Performance Testing:** The system was tested for range, latency, and reliability in open-field conditions.
- **Error Handling:** Tests ensured the system could recover from packet loss or weak signal conditions without system fail.

CHAPTER 7

RESULT

- The LoRa-based wireless telemetry system was successfully designed and implemented using a Raspberry Pi Pico (transmitter) and an Arduino UNO (receiver).
- The transmitter module accurately collected data from multiple sensors:
 - DHT11 – Temperature and humidity
 - BMP280 – Pressure, temperature, and altitude
 - MPU6050 – Acceleration and gyroscope data
 - MQ-2 – Gas concentration (LPG, CO, and Smoke)
- The SX1278 LoRa transceiver operating at 433 MHz provided stable and long-range communication between the transmitter and receiver.
- The receiver module successfully received and displayed telemetry data on a 20x4 I²C LCD, allowing clear and real-time monitoring of flight and environmental parameters.
- The system achieved a communication range of 1.5–2 km in open areas, and 300–500 meters in obstructed environments, demonstrating strong signal performance.
- The data refresh rate was maintained at approximately 1–2 updates per second, ensuring near real-time transmission and display.
- The packet loss rate remained below 2%, indicating high transmission reliability even over long distances.
- The system consumed low power, making it suitable for battery-powered UAV or drone applications.
- The Chirp Spread Spectrum (CSS) modulation technique in LoRa provided excellent noise immunity and interference resistance, resulting in a stable connection.
- All sensor readings were accurate and consistent after calibration, confirming the proper functioning of the hardware and software modules.
- The LCD display showed data in a readable and structured format, improving real-time observation and system usability.

- The overall system performed consistently and reliably during both indoor and outdoor testing, validating its design and objectives.
- The project successfully demonstrated that LoRa communication is a cost-effective, energy-efficient, and long-range alternative to traditional RF modules for telemetry applications.
- The experimental results confirmed that the designed LoRa-based telemetry system successfully meets its intended purpose of long-range, low-power, and real-time data communication.
- The hardware and software integration worked seamlessly, and the system remained stable across various test conditions

CHAPTER 8

FUTURE ENHANCEMENTS

While the developed LoRa-Based Wireless Telemetry System successfully demonstrates real-time data transmission and monitoring between the flight module and ground station, there are several possible improvements and extensions that can make the system more advanced, reliable, and suitable for real-world aerospace or UAV applications.

1. Integration of GPS Module

By adding a GPS receiver module (e.g., NEO-6M), the system can provide real-time position tracking, including latitude, longitude, and altitude. This would allow flight trajectory visualization and logging for analysis and recovery in case of long-distance flights.

2. SD Card Data Logging

Incorporating an SD card module in the transmitter or receiver can enable local data storage of flight telemetry. This allows post-flight analysis of parameters such as altitude, pressure, temperature, and acceleration, even if real-time transmission is interrupted.

3. Graphical Ground Control Interface

A PC-based or mobile Ground Control Station (GCS) can be developed using software such as Python (Tkinter, PyQt) or MATLAB GUI. This interface can display telemetry data in graphs, gauges, or real-time maps instead of just LCD text, improving usability and visualization.

4. Two-Way Communication (Duplex Telemetry)

Currently, the system operates in a unidirectional (one-way) communication mode. Future upgrades can implement bidirectional communication to allow the ground station to send control commands (e.g., motor on/off, parameter reset) to the flight module, enabling full remote control.

5. Advanced Power Management

Integrating power-efficient components or sleep modes for the Raspberry Pi Pico and

sensors can significantly extend battery life, making the system more suitable for long-duration missions or high-altitude balloon experiments.

6. Enhanced Security and Encryption

To prevent unauthorized interception or interference, the LoRa communication can include data encryption algorithms such as AES-128, ensuring data integrity and confidentiality during transmission.

7. Sensor Expansion

Additional sensors such as GPS, barometric pressure, gyroscope, airspeed, or IMU sensors can be added to increase system precision and enable complete environmental and flight dynamics monitoring.

8. Cloud-Based Data Storage and IoT Integration

The receiver station can be connected to the Internet (through Wi-Fi or Ethernet) to upload telemetry data to cloud platforms like ThingSpeak, Blynk, or Firebase. This allows remote monitoring and data access from anywhere.

9. Compact PCB Design

A custom PCB can be designed to integrate all components into a single compact and lightweight board. This improves reliability, reduces wiring errors, and enhances portability for aerial applications.

10. Machine Learning-Based Data Analysis

Collected telemetry data can be processed using machine learning algorithms to predict flight behavior, detect anomalies, or optimize control strategies for autonomous UAV operations.

CHAPTER 9

CONCLUSION

The LoRa-Based Wireless Telemetry System successfully achieves its primary goal of long-range, low-power, and reliable wireless communication between a flight module and a ground control station. By utilizing LoRa SX1278 transceivers, the system overcomes the limitations of traditional RF communication methods, providing robust data transfer even in environments with interference or obstacles.

The integration of sensors such as BMP280, MPU6050, DHT11, and MQ-2 enables the continuous monitoring of critical flight parameters—including altitude, pressure, temperature, humidity, acceleration, and gas concentration. Data is efficiently transmitted by the Raspberry Pi Pico (transmitter) and received by the Arduino UNO (receiver), where it is displayed in real-time on an I²C LCD module.

The project demonstrates the feasibility of implementing a low-cost telemetry solution for small-scale aerospace systems, UAVs, and academic research projects. It provides valuable insight into IoT-based communication, sensor integration, and embedded system design.

Overall, the developed system is:

- Reliable, due to the robustness of LoRa technology.
- Energy-efficient, suitable for battery-powered systems.
- Scalable, allowing the addition of more sensors or modules.
- Educationally valuable, providing a foundation for further innovation in telemetry and remote sensing.

In conclusion, this project proves that LoRa technology is a practical and efficient solution for modern telemetry applications, paving the way for future advancements in long-range wireless communication and flight monitoring systems.

REFERENCE

1. Arduino Documentation, “Arduino UNO Board Overview and Technical Specifications,”[Online].Available: <https://docs.arduino.cc/hardware/uno-rev3>
2. Raspberry Pi Foundation, “Raspberry Pi Pico Datasheet,” Version 1.4, 2022. [Online]. Available: <https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf>
3. HopeRF Electronics, “SX1278 LoRa Transceiver Module Datasheet,” HopeRF, 2018. [Online]. Available: <https://www.hoperf.com/modules/lora/RFM95.html>
4. Bosch Sensortec, “BMP280: Pressure and Temperature Sensor Datasheet,” 2018. [Online]. Available: <https://www.bosch-sensortec.com>
5. InvenSense Inc., “MPU6050: 6-Axis Accelerometer and Gyroscope Datasheet,” Revision3.4,2013.[Online].
Available:<https://invensense.tdk.com/products/motiontracking/6-axis/mpu-6050/>
6. Aosong Electronics, “DHT11 Temperature and Humidity Sensor Datasheet,” Aosong (Guangzhou)Co.Ltd.,2017.[Online]. Available: <https://components101.com>
7. MQ Series Sensors, “MQ-2 Gas Sensor Datasheet,” Hanwei Electronics, 2016. [Online]. Available: <https://www.winsen-sensor.com>
8. IEEE Xplore, “LoRa-based IoT Systems for Remote Sensing and Telemetry Applications,”IEEEAccess,Vol.8,pp.130000–130012,2020.[Online].
Available: <https://ieeexplore.ieee.org>
9. PyCom Documentation, “Using LoRa in MicroPython,” PyCom Ltd., 2022. [Online].Available:<https://docs.pycom.io>

APPENDIX

Source Code:

TRANSMITTER:

```
import machine

import time

import dht

import math

from sx127x import SX127x

import bmp280

from mpu6050 import MPU6050

# -----

# LoRa Setup (for Pico/ESP32)

# -----

spi = machine.SPI(0, baudrate=5000000, polarity=0, phase=0,
sck=machine.Pin(18), mosi=machine.Pin(19), miso=machine.Pin(16))

cs = machine.Pin(17, machine.Pin.OUT) # LoRa CS

reset = machine.Pin(20, machine.Pin.OUT) # LoRa RESET

dio0 = machine.Pin(21, machine.Pin.IN) # LoRa DIO0


lora = SX127x(spi, cs, reset, dio0, frequency=433E6)

print("LoRa Transmitter Ready")

# -----

# Sensor Setup

# -----

dht_sensor = dht.DHT11(machine.Pin(15))

i2c = machine.I2C(0, scl=machine.Pin(1), sda=machine.Pin(0))

bmp = bmp280.BMP280(i2c)
```

```

mpu = MPU6050(i2c)
mq2 = machine.ADC(26)
RL = 10.0
Ro = 10.0
# -----
# MQ-2 Calibration
# -----
def calibrate_mq2():
    print("Calibrating MQ-2 in clean air...")
    val = 0
    for i in range(50):
        val += mq2.read_u16()
    time.sleep(0.1)
    val = val / 50
    voltage = (val / 65535) * 3.3
    Rs = (3.3 - voltage) * RL / voltage
    Ro = Rs / 9.83
    print("MQ-2 Calibration done. Ro =", Ro)
    return Ro
Ro = calibrate_mq2()
# -----
# MQ-2 Gas Calculation
# -----
def read_gas(Ro):
    val = mq2.read_u16()
    voltage = (val / 65535) * 3.3
    Rs = (3.3 - voltage) * RL / voltage
    ratio = Rs / Ro

```

```

gases = {
"LPG": math.pow(10, ((math.log10(ratio) - 2.3) / -0.45)),
"CO": math.pow(10, ((math.log10(ratio) - 2.7) / -0.37)),
"Smoke": math.pow(10, ((math.log10(ratio) - 2.0) / -0.36))
}
return gases

# -----
# Main Loop — Send via LoRa
# -----
while True:
try:
# Read sensors
dht_sensor.measure()
temp_dht = dht_sensor.temperature()
hum = dht_sensor.humidity()
temp_bmp = bmp.temperature
pressure = bmp.pressure
altitude = bmp.altitude
accel = mpu.get_accel_data()
gyro = mpu.get_gyro_data()
gases = read_gas(Ro)

# Prepare message
message = "DHT: {}C {}% | BMP: {}C {}hPa {}m | MPU Acc:{} Gyro:{} | MQ2 LPG:{}
CO:{} Smoke:{}".format(
temp_dht, hum,
round(temp_bmp,2), round(pressure,2), round(altitude,2),
accel, gyro,

```

```

round(gases["LPG"],2), round(gases["CO"],2), round(gases["Smoke"],2)
)
# Send via LoRa
lora.begin_packet()
lora.println(message)
lora.end_packet()
print("Sent:", message)
time.sleep(2)
except Exception as e:
print("Error:", e)
time.sleep(2)

```

RECEIVER:

```

#include <SPI.h>
#include <LoRa.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// -----
// LoRa Setup
// -----
#define LORA_SS 5 // LoRa CS
#define LORA_RST 14 // LoRa RESET
#define LORA_DIO0 26 // LoRa DIO0
#define LORA_FREQ 433E6 // Frequency
// -----
// LCD Setup (20x4)
// -----
LiquidCrystal_I2C lcd(0x27, 20, 4); // I2C address 0x27, 20 columns, 4 rows

```

```

void setup() {
  Serial.begin(115200);
  while (!Serial);
  // LCD init
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("LoRa Receiver Ready");
  // LoRa init
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
  if (!LoRa.begin(LORA_FREQ)) {
    Serial.println("Starting LoRa failed!");
    lcd.setCursor(0, 1);
    lcd.print("LoRa Failed");
    while (1);
  }

  Serial.println("LoRa Receiver Ready");
}

void loop() {
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    String received = "";
    while (LoRa.available()) {
      char c = (char)LoRa.read();
      received += c;
    }
  }
}

```



```

Serial.print("Received: ");
Serial.println(received);
// Split received message by "|"
String lines[4];
int start = 0;
for (int i = 0; i < 4; i++) {
int idx = received.indexOf('|', start);
if (idx == -1) {
lines[i] = received.substring(start);
} else {
lines[i] = received.substring(start, idx);
}
lines[i].trim();
if (lines[i].length() > 20) lines[i] = lines[i].substring(0, 20);
start = idx + 1;
}
// Display each line on 20x4 LCD
lcd.clear();
for (int i = 0; i < 4; i++) {
lcd.setCursor(0, i);
lcd.print(lines[i]);
}
}
}

```